

Transactions

Remy Wang

April 2026

So far we've been treating DBs as inanimate objects: they hold data, we can look at the data, and that's it. But SQL databases are so popular because they deal with *changes* very well. Let's see some examples.

I'm in a good mood today and decided to give everyone free points!
I vibed a quick python script to do this:

UID	score
1	39
2	24
3	40
4	35
...	...

I'm in a good mood today and decided to give everyone free points!
I vibed a quick python script to do this:

UID	score
1	39
2	24
3	40
4	35
...	...

Uh oh! My computer crashed halfway through, and I don't know who still haven't got the free points!

You don't like that, because some of you got the points and some didn't. You would be fine if either none of you get the points or all of you did.

Let's try another experiment. This time, your fate is in your own hands: steal points from your enemies! Subtract some points from someone and add it to your own. Here's the Google sheet, go!

Well, that was a mess. The total score no longer add up, meaning our DB is not *consistent*!

The problem is that you were *interfering* with each other!

Well, that was a mess. The total score no longer add up, meaning our DB is not *consistent*!

The problem is that you were *interfering* with each other!

One solution is to go *one at a time*, so that every change is made in *isolation*. In other words, we *serialize* the changes.

Another solution is we first claim a *lock* on a piece of data before changing it. This time, wait a few seconds before editing a cell, and only edit it if no one else is trying to edit.

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- ▶ **Atomic:** either all or none of the actions are applied (all or nothing)

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- ▶ **Atomic**: either all or none of the actions are applied (all or nothing)
- ▶ **Consistent**: after a tx finishes, the DB is in a consistent state

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- ▶ **Atomic:** either all or none of the actions are applied (all or nothing)
- ▶ **Consistent:** after a tx finishes, the DB is in a consistent state
- ▶ **Isolated:** txs do not interfere with each other

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- ▶ **Atomic:** either all or none of the actions are applied (all or nothing)
- ▶ **Consistent:** after a tx finishes, the DB is in a consistent state
- ▶ **Isolated:** txs do not interfere with each other
- ▶ **Durable:** once a tx finishes, its effect is kept forever

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- ▶ **Atomic:** either all or none of the actions are applied (all or nothing)
- ▶ **Consistent:** after a tx finishes, the DB is in a consistent state
- ▶ **Isolated:** txs do not interfere with each other
- ▶ **Durable:** once a tx finishes, its effect is kept forever

Formally, a group of actions on a DB is called a *transaction*. DBs like SQLite guarantee transactions (tx) are:

- ▶ **Atomic**: either all or none of the actions are applied (all or nothing)
- ▶ **Consistent**: after a tx finishes, the DB is in a consistent state
- ▶ **Isolated**: txs do not interfere with each other
- ▶ **Durable**: once a tx finishes, its effect is kept forever

This is known as *ACID* on the street.

Note that *consistent* is a high-level property, usually achieved with the other 3.

- ▶ When only some of you got the extra credit, our tx was not *atomic*
- ▶ When you were stealing points and had conflicts, you were not *isolated*, leaving the DB *inconsistent*
- ▶ We haven't see *durable* yet, but if the computer dies before writing data to disk, the data would be lost

- ▶ When only some of you got the extra credit, our tx was not *atomic*
- ▶ When you were stealing points and had conflicts, you were not *isolated*, leaving the DB *inconsistent*
- ▶ We haven't see *durable* yet, but if the computer dies before writing data to disk, the data would be lost

Luckily, the DB handles all these for us!

But it's still important to understand what's going on, in case anything goes sideways.

There are several different ways to ensure *atomicity*. The main idea is to a tx to be explicitly COMMITted:

```
BEGIN TRANSACTION;
```

```
SELECT ...;
```

```
INSERT ...;
```

```
COMMIT;
```

You can think of each tx as making a local copy of the DB, and any operation in that tx only acts upon the local copy, until the tx is committed which writes the DB to the global one.

Try running the following concurrently in 2 SQLite sessions¹:

<i>--</i>	<i>SESSION 1</i>	<i>SESSION 2</i>
	<code>create table r (x int);</code>	
	<code>insert into r values (1);</code>	
		<code>select * from r;</code>
	<code>begin transaction;</code>	
	<code>insert into r values (2);</code>	
		<code>select * from r;</code>
	<code>commit;</code>	
		<code>select * from r;</code>

You'll see the change made within the tx does not take effect until after COMMIT.

¹Run `sqlite3 test.db` in 2 different terminal tabs.

The easiest way to guarantee *isolation* is to *serialize* the transactions – running them one at a time. This is pretty much what SQLite does!² So why don't we just leave it at that?

²SQLite serializes writes, but allow concurrent reads.

The easiest way to guarantee *isolation* is to *serialize* the transactions – running them one at a time. This is pretty much what SQLite does!² So why don't we just leave it at that?

Because performance. When we serialized our score updates, we all had to wait in line, even though we could have used some parallelism.

²SQLite serializes writes, but allow concurrent reads.

So on one hand, serialization guarantees consistency but is slow

On the other hand, concurrency is fast but is inconsistent

The key idea of TX processing is to preserve consistency while improving concurrency, which is usually done by giving the *illusion* of serial execution while running concurrently. Magic!

First off, if 2 TX touch different data, they can of course run concurrently:

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

Here R(A) means reading a DB item A, and W(A) writes to A. You can think of a DB item as a row, but it depends on the context.

The table here shows the *schedule*³ of the txs which is a record of actions taken over time.

³Formally, a schedule is a *partial ordering* of the actions.

We say a schedule is *serial* if all actions are strictly ordered:

T ₁	T ₂
R(A)	
W(A)	
	R(B)
	W(B)

In our example, the concurrent schedule happens to be *equivalent* to the serial one, because the TX involve independent items:

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

T ₁	T ₂
R(A)	
W(A)	
	R(B)
	W(B)

Formally, two TX are *equivalent* if they produce the same result no matter what is read or written. . . . And we say the concurrent schedule is *serializable*, i.e., it is equivalent to some serial schedule.

The main idea of DBs is that we want to allow **serializable**, yet concurrent **transaction schedules** like the above.⁴

⁴Make sure you know what the bold words mean by now!

Note that two serial schedules are not necessarily equivalent:

T_1	T_2
$A = 2 * A$	
	$A = 2 + A$

T_1	T_2
	$A = 2 + A$
$A = 2 * A$	

If we start with $A = 1$, the first schedule would result in $A = 4$, while the second in $A = 6$.

Note that two serial schedules are not necessarily equivalent:

T ₁	T ₂
A = 2*A	
	A = 2+A

T ₁	T ₂
	A = 2+A
A = 2*A	

If we start with $A = 1$, the first schedule would result in $A = 4$, while the second in $A = 6$.

For a schedule to be *serializable*, it only needs to be equivalent to *some* serial one, but we don't know which one.

There's also the notion of *strict serializability*, which basically says “first come, first serve”:

- ▶ A schedule is strict-serializable if it's equivalent to a serial schedule that runs the TXs in the same order they arrive.

This can be important for e.g. ticket sales.

To check if a schedule is serializable, we first need to know how to check if 2 schedules are equivalent.

To do that, we check if we can *reorder* one into another, while avoiding *conflicts*:

T ₁	T ₂
R(A)	R(B)
W(A)	W(B)

T ₁	T ₂
R(A)	
	R(B)
W(A)	
	W(B)

T ₁	T ₂
R(A)	
W(A)	
	R(B)
	W(B)

1. Because reading data never causes conflict, we are free to reorder R(A) and R(B) any way we like.

To check if a schedule is serializable, we first need to know how to check if 2 schedules are equivalent.

To do that, we check if we can *reorder* one into another, while avoiding *conflicts*:

<u>T₁</u>	<u>T₂</u>
R(A)	R(B)
W(A)	W(B)

<u>T₁</u>	<u>T₂</u>
R(A)	
	R(B)
W(A)	
	W(B)

<u>T₁</u>	<u>T₂</u>
R(A)	
W(A)	
	R(B)
	W(B)

1. Because reading data never causes conflict, we are free to reorder R(A) and R(B) any way we like.
2. Since writing to different items do not conflict, we can also reorder W(A) and W(B).

To check if a schedule is serializable, we first need to know how to check if 2 schedules are equivalent.

To do that, we check if we can *reorder* one into another, while avoiding *conflicts*:

<u>T₁</u>	<u>T₂</u>
R(A)	R(B)
W(A)	W(B)

<u>T₁</u>	<u>T₂</u>
R(A)	
	R(B)
W(A)	
	W(B)

<u>T₁</u>	<u>T₂</u>
R(A)	
W(A)	
	R(B)
	W(B)

1. Because reading data never causes conflict, we are free to reorder R(A) and R(B) any way we like.
2. Since writing to different items do not conflict, we can also reorder W(A) and W(B).
3. There's also no conflict between W(A) and R(B), so we swap one more time and the schedule is serial.

In general, two actions are in conflict if they:

- ▶ operate on the same DB item
- ▶ one of them is a write

For example, the following schedules are not equivalent, because they cannot be reordered into each other:

T ₁	T ₂
R(A)	
	R(A)
W(A)	
	W(A)

T ₁	T ₂
R(A)	
W(A)	
	R(A)
	W(A)

Indeed, the first schedule is not *serializable*, and the two TXs are attempting to change the same item at the same time.

For a concrete example:

T_1	T_2
$x=R(A)$	
$y=2x$	$u=R(A)$
$W(A)=y$	$v=2u$
	$W(A)=v$

If the TXs run in sequence, we would multiply A by 4, but the schedule above would only multiply by 2.

In other words, *no serial schedule* would have produced the same result.

This gives us an algorithm to check for serializability:

- ▶ First compute all possible serial schedules involving the TXs
- ▶ For each serial schedule, check if it can be reordered into the given one

But this is hopelessly slow: there are exponentially many serial schedules, and each can have exponentially many reorderings.

We can check serializability with a powerful tool called the *precedence graph*. Consider the following schedule:

T ₁	T ₂	T ₃
R(B)	R(A)	
	W(A)	
W(B)		R(A)
	R(B)	
	W(B)	

The graph has a node per transaction, and an edge $T_i \rightarrow T_j$ if there's a pair of actions $a_i \in T_i$ and $a_j \in T_j$ s.t.:

- ▶ a_i occurs no later than a_j
- ▶ they conflict with each other

$$T_1 \rightarrow T_2 \rightarrow T_3$$

Let's also check the schedule that we know is not serializable:

T_1	T_2
$x=R(A)$	
$y=2x$	$u=R(A)$
$W(A)=y$	$v=2u$
	$W(A)=v$

Let's also check the schedule that we know is not serializable:

T_1	T_2
$x=R(A)$	
$y=2x$	$u=R(A)$
$W(A)=y$	$v=2u$
	$W(A)=v$

Its precedence graph has a cycle:

$$T_1 \rightleftarrows T_2$$

A schedule is (conflict-)serializable iff its precedence graph has no cycle

A schedule is (conflict-)serializable iff its precedence graph has no cycle

Intuitively, an edge $T_i \rightarrow T_j$ means there's a pair actions a_i, a_j that cannot be reordered due to a conflict, so a_i must occur first in any reordering, meaning some part of T_i must happen before T_j .

But if there's another edge $T_j \rightarrow T_i$, some part of T_j would also happen before T_i , which is impossible in a serial schedule.

That was a lot of new concepts to learn! Let's review:

- ▶ We want to guarantee ACID when the DB is modified

That was a lot of new concepts to learn! Let's review:

- ▶ We want to guarantee ACID when the DB is modified
- ▶ So we group actions into TXs as units

That was a lot of new concepts to learn! Let's review:

- ▶ We want to guarantee ACID when the DB is modified
- ▶ So we group actions into TXs as units
- ▶ Serial execution would lead to ACID, but slow

That was a lot of new concepts to learn! Let's review:

- ▶ We want to guarantee ACID when the DB is modified
- ▶ So we group actions into TXs as units
- ▶ Serial execution would lead to ACID, but slow
- ▶ We want serializable concurrent schedules

That was a lot of new concepts to learn! Let's review:

- ▶ We want to guarantee ACID when the DB is modified
- ▶ So we group actions into TXs as units
- ▶ Serial execution would lead to ACID, but slow
- ▶ We want serializable concurrent schedules
- ▶ Which are schedules equivalent to some serial ones

That was a lot of new concepts to learn! Let's review:

- ▶ We want to guarantee ACID when the DB is modified
- ▶ So we group actions into TXs as units
- ▶ Serial execution would lead to ACID, but slow
- ▶ We want serializable concurrent schedules
- ▶ Which are schedules equivalent to some serial ones
- ▶ Which we can check with the precedence graph

The way things are going might make you think this is how DB systems work:

- ▶ A bunch of transaction actions get submitted to the DB

The way things are going might make you think this is how DB systems work:

- ▶ A bunch of transaction actions get submitted to the DB
- ▶ Everytime the DB sees a new action, it checks serializability

The way things are going might make you think this is how DB systems work:

- ▶ A bunch of transaction actions get submitted to the DB
- ▶ Everytime the DB sees a new action, it checks serializability
- ▶ Which is done by constructing the precedence graph and checking for cycles

The way things are going might make you think this is how DB systems work:

- ▶ A bunch of transaction actions get submitted to the DB
- ▶ Everytime the DB sees a new action, it checks serializability
- ▶ Which is done by constructing the precedence graph and checking for cycles
- ▶ If there's a cycle, ???

The way things are going might make you think this is how DB systems work:

- ▶ A bunch of transaction actions get submitted to the DB
- ▶ Everytime the DB sees a new action, it checks serializability
- ▶ Which is done by constructing the precedence graph and checking for cycles
- ▶ If there's a cycle, ???

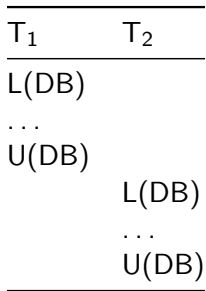
The way things are going might make you think this is how DB systems work:

- ▶ A bunch of transaction actions get submitted to the DB
- ▶ Everytime the DB sees a new action, it checks serializability
- ▶ Which is done by constructing the precedence graph and checking for cycles
- ▶ If there's a cycle, ???

The problem with this is that, if we check serializability *after the fact*, it might be too late.

In reality, the DB guarantees serializable schedule *by construction* by using locks.⁵

The simplest approach is locking the whole DB every time:



Each tx takes the lock before it starts, and unlocks after it's done. This would result in a *serial* schedule.

⁵There *are* also approaches that check after-the-fact, covered later.

But locking the whole DB is too conservative! What if we “only lock what we need”?

<u>T₁</u>	<u>T₂</u>
L(A); R(A)	
W(A); U(A)	
	L(A), R(A)
	W(A), U(A)
	L(B), R(B)
	W(B), U(B)
L(B); R(B)	
W(B); U(B)	

The schedule is not serializable! Try out a concrete example to see.

Again, we're facing a dilemma:

- ▶ locking everything is correct, but slow
- ▶ locking individual items is fast, but wrong

Again, we're facing a dilemma:

- ▶ locking everything is correct, but slow
- ▶ locking individual items is fast, but wrong

The middle ground taken by most DBs is **2-phase locking**(2PL):

- ▶ an item must be locked before being read/written
- ▶ within each tx, all locks must precede all unlocks

This still allows *some* degree of concurrency:

T_1	T_2
L(A),R(A),W(A)	L(B),R(B),W(B),U(B)
L(B),R(B),W(B),U(A),U(B)	

Here, because the tx need different items in the beginning, they can run in parallel; then once T_2 is done with B, T_1 can lock it.

The main result in DB transactions is:

Theorem: 2PL guarantees serializability

Some notations first:

- ▶ Let $i \rightarrow j$ denote an edge in the graph
- ▶ Let $L_i(X), U_i(X)$ denote tx i locking/unlocking X

Some notations first:

- ▶ Let $i \rightarrow j$ denote an edge in the graph
- ▶ Let $L_i(X), U_i(X)$ denote tx i locking/unlocking X

We'll also use this lemma:

Lemma: and edge $i \rightarrow j$ means the schedule contains a pattern $U_i(X), \dots, L_j(X)$

This is because the conflict edge means T_i acts on X before T_j , and for that to happen, T_i must unlock X before T_j locks it.

Now, suppose we have a cycle involving 3 tx:

$$1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1$$

Then, we must have this pattern:

$$U_1, L_2, U_2, L_3, U_3, L_1$$

But this violates 2PL, contradiction!

So far, we have assumed every tx will eventually COMMIT, in which case 2PL is sufficient.

But in many cases a tx may actually get cancelled, e.g. if your card declines when buying tickets.

T_1	T_2
L(A); R(A); W(A); U(A)	L(A); R(A); W(A); U(A); COMMIT
ROLLBACK	

Because tx is atomic, a cancelled tx needs to be undone. But here it's too late! The effect of T_1 is taken in by T_2 and COMMITted into the DB.

To fix this, we need **strict 2PL**, which additionally requires unlock to happen exactly at commit/rollback time.⁶

T₁

T₂

L(A); R(A); W(A); ROLLBACK, U(A)

L(A); R(A); W(A); U(A); COMMIT

⁶If you find “exactly the same time” confusing, think of it as unlocking *after* commit/rollback.

Another issue with locking is *deadlocks*:

|T₁: W(A),W(B)|T₁: W(B),W(C)|T₁: W(C),W(A)| |L(A),W(A)|||
||L(B),W(B)|| ||L(C),W(C)| |L(B)?|L(C)?|L(A)?|

Here, after each thread grabs its lock, they then want each other's lock for the next action. But because of 2PL, they can't release any locks. Impasse!

To resolve the deadlock, someone has to back up.

First, to detect deadlocks, we need to construct the *wait-for* graph, where an edge $i \rightarrow j$ means i is waiting for the lock held by j .

In our example, we have $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1$ – a cycle!

Whenever there's a cycle in the wait-for graph, there's a deadlock.

In that case, we must rollback a tx to break the cycle.

Although 2PL allows concurrency, it's usually still not fast enough.

Serializability inherently requires some sequential-ness.

In practice, people sacrifice serializability for better performance by adopting **weak isolation levels**.

We will look at one such level – snapshot isolation.

The idea is simple: each tx acts on a DB snapshot, then merge at COMMIT:

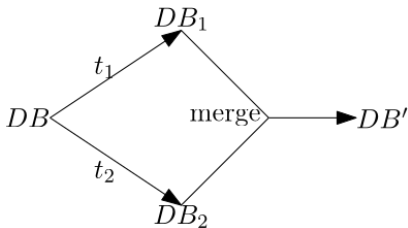


Figure 1: Snapshot Isolation

The merge succeeds as long as there's no *write-write* conflict.

SI is attractive because it is intuitively *atomic* and *isolating*:

- ▶ Each tx operates on own snapshot, and either COMMIT or ROLLBACK
- ▶ Local operations *cannot* interfere with each other

SI is attractive because it is intuitively *atomic* and *isolating*:

- ▶ Each tx operates on own snapshot, and either COMMIT or ROLLBACK
- ▶ Local operations *cannot* interfere with each other

Surprisingly, snapshot isolation does *not* guarantee serializability!

The reason is that it breaks *consistency*.

Suppose you have 2 tx:

T_1	T_2
$x=R(A)$	$y=R(B)$
$x=2x$	$y=2y$
$W(B)=x$	$W(A)=y$

In any serial ordering, we would end up with one value being twice of the other.

But under SI, because the writes don't conflict, the tx's run concurrently and end up with $A=B$.

For another example, suppose you're working on a project with your partner.

- ▶ Your codebase implements `def f(): return 1`

For another example, suppose you're working on a project with your partner.

- ▶ Your codebase implements `def f(): return 1`
- ▶ You add `def g(): return f() + 1`

For another example, suppose you're working on a project with your partner.

- ▶ Your codebase implements `def f(): return 1`
- ▶ You add `def g(): return f() + 1`
- ▶ Your friend changes `def f(): return 2`

For another example, suppose you're working on a project with your partner.

- ▶ Your codebase implements `def f(): return 1`
- ▶ You add `def g(): return f() + 1`
- ▶ Your friend changes `def f(): return 2`

For another example, suppose you're working on a project with your partner.

- ▶ Your codebase implements `def f(): return 1`
- ▶ You add `def g(): return f() + 1`
- ▶ Your friend changes `def f(): return 2`

Now although there's no git conflict, your code is broken!

This is known as *merge skew*, and can happen even if you add tests.

For another example, suppose you're working on a project with your partner.

- ▶ Your codebase implements `def f(): return 1`
- ▶ You add `def g(): return f() + 1`
- ▶ Your friend changes `def f(): return 2`

Now although there's no git conflict, your code is broken!

This is known as *merge skew*, and can happen even if you add tests.

There have been work fixing this, resulting the so-called *serializable snapshot isolation* (SSI).

What's also surprising is that people seem to use *even weaker* levels of isolation in practice.

For example, PostgreSQL defaults to Read Committed.

There's research showing anything weaker than *serializable* is vulnerable to attacks, but no one really knows why things just don't break!